

Machine Learning Lab Schedule

Experiments	Content	Week
 1	KNN Algorithm for Iris Flower Classification	Week 6 Friday: 6:45-10:05 PM Room T2510 
2	Classification of Wine Dataset Using PCA and SVM Algorithms	Week 8 Thursday: 10:30-12:15AM Thursday: 4:00-17:45 PM Room T2510 
3	CNN for MNIST Digit Recognition	Week 10 Wednesday: 4:00-17:45 PM Thursday: 4:00-17:45 PM Room T2510 



Experiment 2

Classification of Wine Dataset Using PCA and SVM Algorithms

2025

Objective

- Master PCA for feature reduction to simplify Wine dataset
- Apply SVM to classify PCA-reduced Wine dataset
- Learn PCA, SVM implementation in Python & model evaluation
- Understand PCA-SVM synergy in classification & evaluate performance



Environment

➤ Python environment:

- Anaconda & Jupyter Notebook

➤ Import required libraries:



Notebook

```
# For numerical computations  
import numpy as np
```

```
# For splitting the dataset  
from sklearn.model_selection import train_test_split
```

```
# For data manipulation and analysis  
import pandas as pd
```

```
# Used for standardizing feature  
from sklearn.preprocessing import StandardScaler
```

```
# For visualizations  
import matplotlib.pyplot as plt
```

```
# To measure execution time  
import time
```

```
# Import the SVM module from scikit-learn  
from sklearn import svm
```

```
#For model evaluation  
from sklearn import metrics
```

```
# Import the PCA class from the decomposition module  
from sklearn.decomposition import PCA
```

Experiment Tasks

1. Dataset Loading

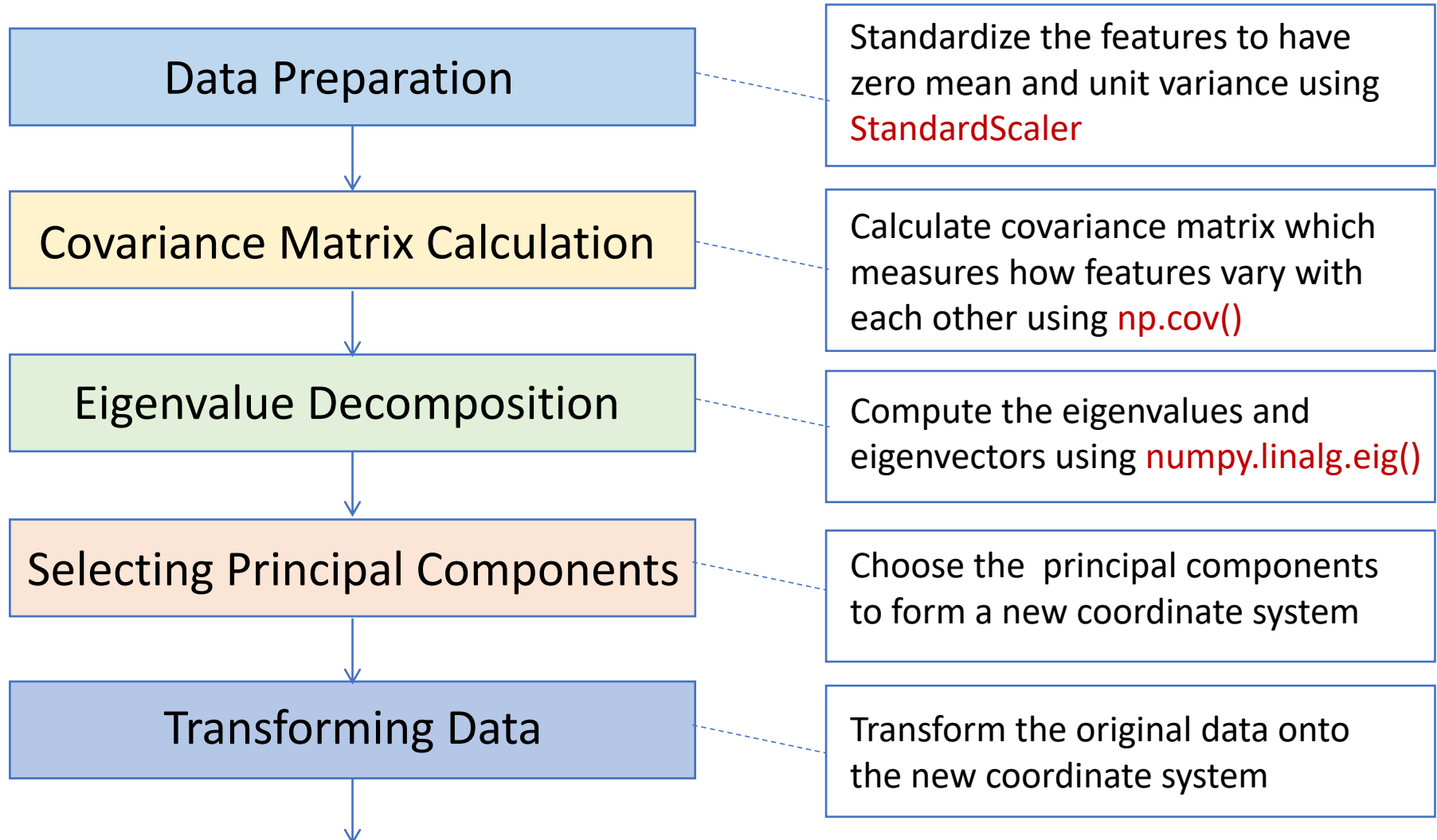
The wine dataset has **178 samples** of chemical analysis results, described by **13 attributes** of chemical compound concentrations. It aims to classify wines into **three cultivars** based on these measurements.



	Class label	Alcohol	Malic acid	Ash	Alcalinity of ash	Magnesium	Total phenols	Flavanoids	Nonflavanoid phenols	Proanthocyanins	Color intensity	Hue	OD280/OD315 of diluted wines	Proline
0	1	14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065
1	1	13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050
2	1	13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185
3	1	14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480
4	1	13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735

Experiment Tasks

2. PCA Dimensionality Reduction



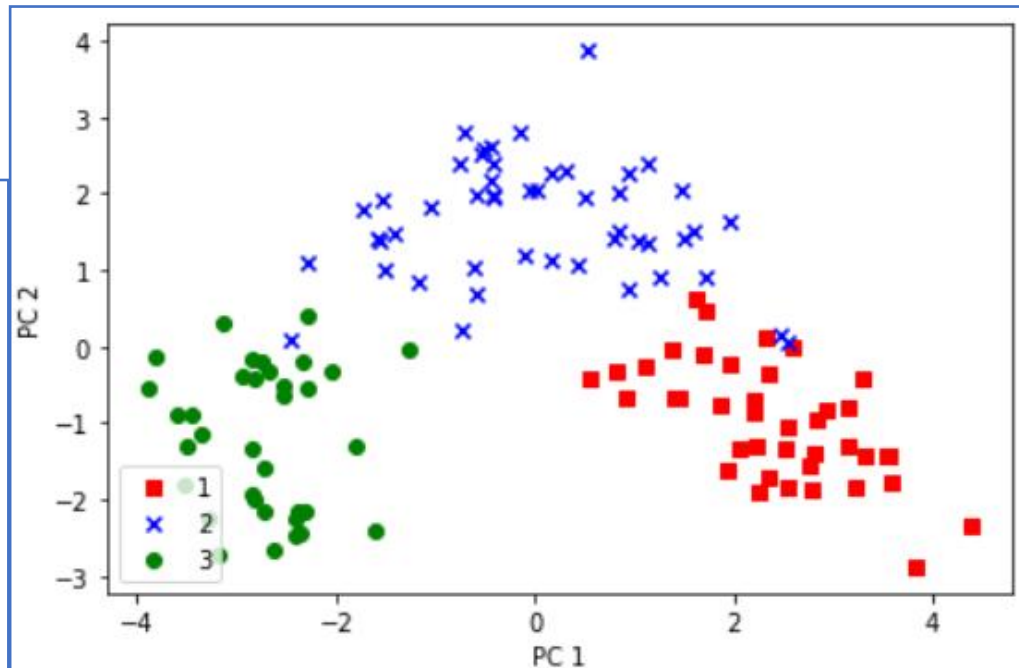
Experiment Tasks

2. PCA Dimensionality Reduction

↓

Visualization

By above steps, PCA **reduces the complexity of the dataset**, making it easier to visualize, analyze, and often improving the performance of machine learning algorithms.

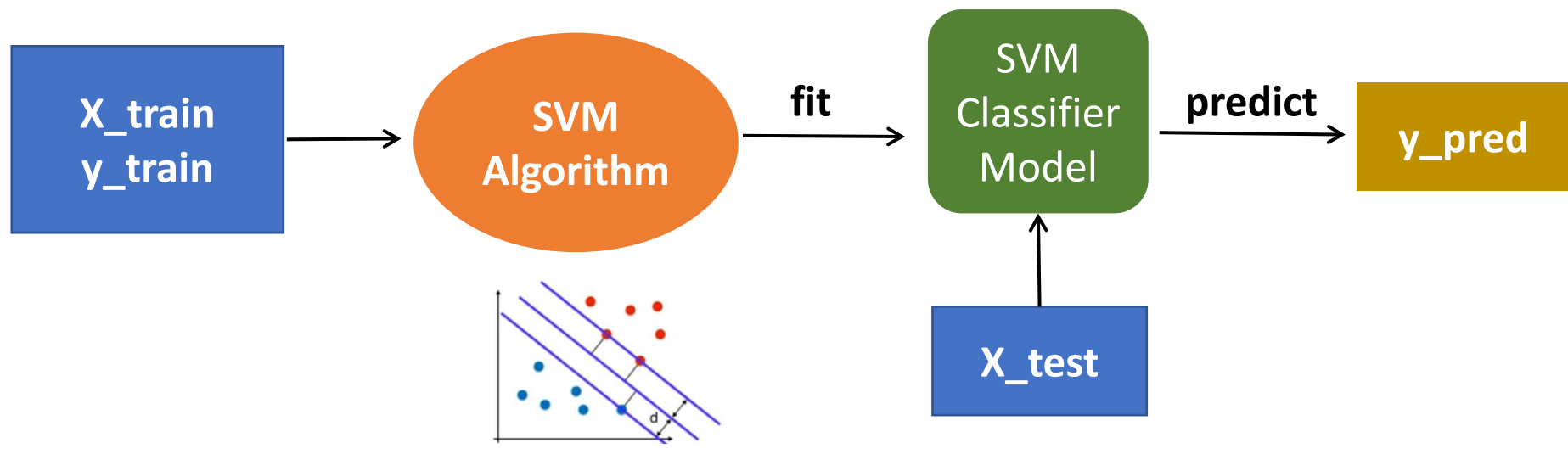


Reduce data from 13D to 2D

Experiment Tasks

3. Model Training

- Split the dataset into training and testing dataset
- Initialize and the SVM model with kernel='linear'
- Pass the training dataset to SVM algorithm to train a model
- Pass the testing data to the trained model to predict the outcome



Experiment Tasks

4. Model Evaluation

- Conduct comparative experiments by training SVM models using different training sets (Original vs. PCA-Transformed Training Data)
- Evaluate the model's accuracy and performance
- Hyperparameter tuning:
 - kernel = 'linear'/'poly'/'rbf'/'sigmoid'
 - C (Regularization Parameter): default is 1.0

Experiment Tasks

5. PCA in Scikit-learn

- Scikit-learn offers a simplified implementation of PCA via the "sklearn.decomposition.PCA" class.

```
# Section: Training and Evaluating SVM With PCA in Scikit-learn
# Import the PCA class from the decomposition module of the scikit-learn library
from sklearn.decomposition import PCA

# Initialize PCA for dimensionality reduction, keeping only 2 principal components
pca = PCA(n_components=2)

# Apply PCA transformation to standardized training dataset
X_train_pca = pca.fit_transform(X_train_std)

# Apply the same PCA transformation to standardized testing dataset (only transform)
X_test_pca = pca.transform(X_test_std)

# Initialize a Support Vector Machine (SVM) classifier with linear kernel, regularization parameter C=1,
# and a random state for reproducibility
clf = svm.SVC(kernel='linear', C=1, random_state=42)

# Train the SVM classifier using the transformed training dataset
clf.fit(X_train_pca, y_train)

# Predict labels for the transformed testing dataset
y_pred = clf.predict(X_test_pca)

# Calculate and print the accuracy score comparing predicted labels with actual labels of the testing dataset
print(metrics.accuracy_score(y_test, y_pred))
```



THANK YOU!